

Topic 6.5

6.5: Derived Classes and Operator <<

Notice that for our `Thing`, `Person`, and `Politician` classes we have defined `print()` member functions, and we do not define `operator<<`.

The `print()` member functions are hierarchical. That is, each `print()` invokes the `print()` member function of its base class, then prints out the additional information associated with the derived class.

It is not possible to directly construct `operator<<` to be hierarchical, because they are *friend* functions, not member functions. We can still define these operators for each class, but each overloaded `operator<<` must output all the information for the object, included the information associated with each base class:

```
ostream& operator<< ( ostream& str, Thing& t )
{
    str << "name:" << t.name;
    return str;
}
ostream& operator<< ( ostream& str, Person& p )
{
    str << "name:" << p.name;
    str << " age:" << p.age;
    return str;
}
ostream& operator<< ( ostream& str, Politician& p )
{
    str << "name:" << p.name;
    str << " age:" << p.age;
    str << " party:" << p.party;
    return str;
}
```

This is somewhat undesirable, since any change in a base class may require changes to all derived base class `operator<<` functions. After learning about *virtual* member functions, we will see a way to achieve this in a (slightly kludgy) hierarchical manner.

The following program illustrates this barely intelligible statement (presented earlier and repeated here for convenience):

If a base class defines multiple versions of some member function (each with a unique signature), and a derived class overrides only one of them, the remaining functions will not be visible to objects of the derived class.

This is illustrated with the following simple example:

```
// MODULE:      mammal1.cpp
// PROGRAMMER:  Gary J. Blair
// LANGUAGE:    C++
// DATE:       96/04/11
//
// PURPOSE:
//   Illustrate name hiding.
//

#include <iostream>
#include <stdlib.h>

using namespace std;

class Mammal
{
public:
    void Move() const { cout << "Mammal moves 1
step.\n"; };
    void Move( int distance ) const
    { cout << "Mammal moves " << distance
      << " steps.\n"; };

protected:
    int age;
    int weight;
};

class Dog : public Mammal
{
public:
    void Move() const { cout << "Dog moves 2 steps.\n"; };

    // You may receive a compiler warning at this
    // point, indicating that a function is being
    // hidden. Specifically, the function
    // void Mammal::Move( int distance ) cannot
    // be applied to a Dog object. When you
    // override a function name, you must override
    // all versions of the function, else the ones
    // not specifically overridden will be hidden
    // in the derived class.
};

int main(void)
{
    Mammal big_animal;
    Dog    spot;

    big_animal.Move();
    big_animal.Move(7);
    spot.Move();
    // spot.Move(12);
    // This would cause a compilation error
    // if not commented out.

    return 0;
}
```

Output from program mammal1.cpp:

```
Mammal moves 1 step.  
Mammal moves 7 steps.  
Dog moves 2 steps.
```

Topic 6.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-6-dot-6>)